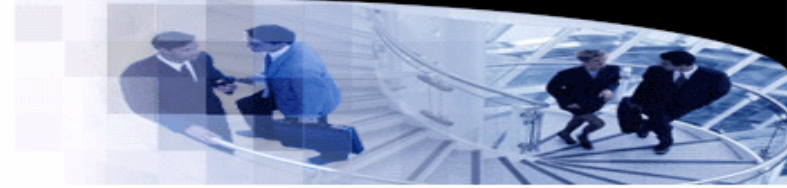




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

Lecture 1. Introduction

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

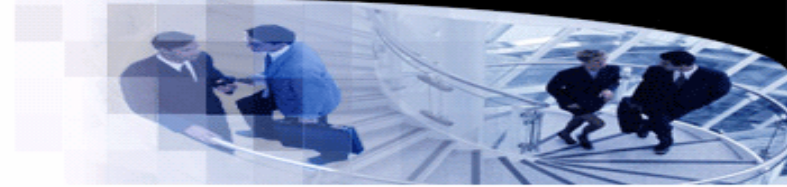
Motivation

- **Example 1**

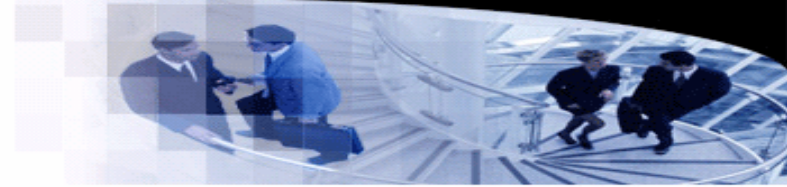
- What is the output of the following C++ program?

```
#include <stdio.h>

int main() {
    signed char a;
    unsigned int b;
    a = -20;
    b = 10;
    if( a < b )
        printf( "a < b\n" );
    else
        printf( "a >= b\n" );
    return 0;
}
```



Motivation



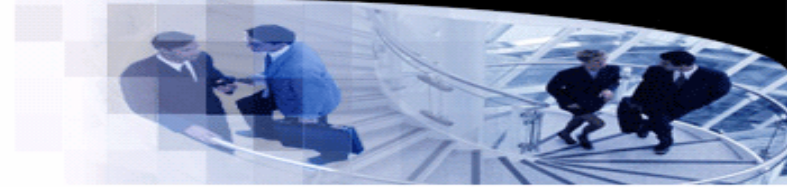
- Example 2

- Given the two following programs, which one is better?

```
double Girth1(double r) {  
    return 6.2831852 * r;  
}
```

```
double Girth2(double r) {  
    const double PI = 3.1415926;  
    return 2 * PI * r;  
}
```

Motivation



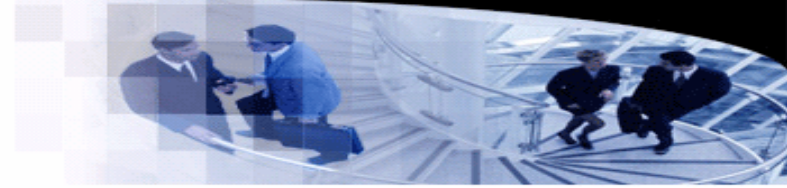
- Example 3

- What are the differences between these two versions of atoi?

```
int atoi(char *str) {  
    ...  
    return ...;  
}
```

```
int atoi(const char *str) {  
    ...  
    return ...;  
}
```

Course Overview



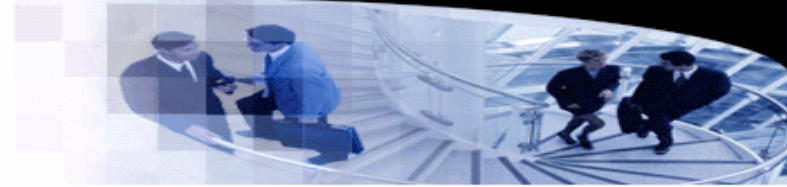
- Goals

- To learn how to **design** and **implement** a programming language.

- Topics

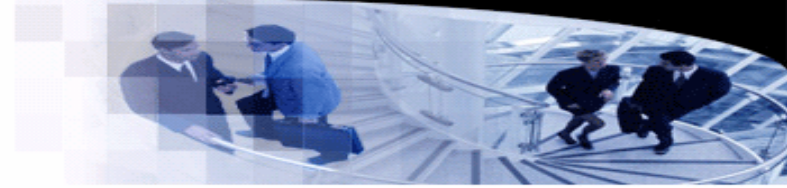
- Functional programming – a declarative programming paradigm.
 - Translation between programs.
 - Program analysis and optimizations.

Course Information



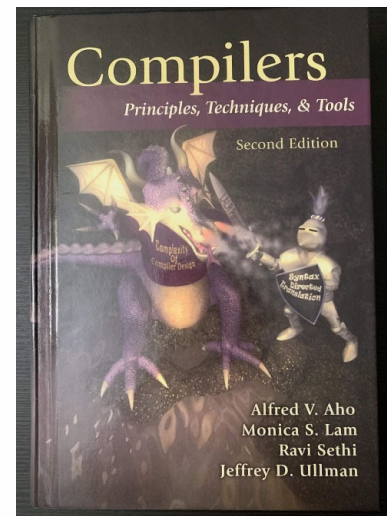
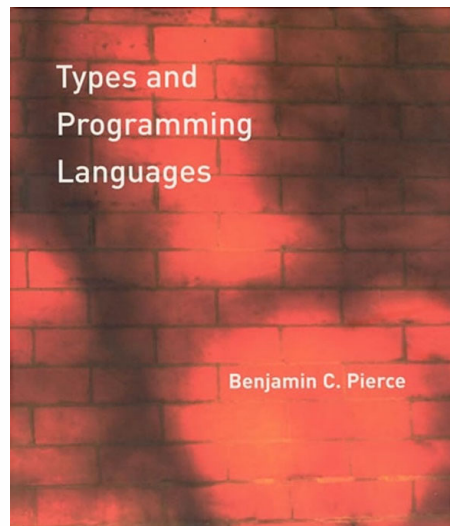
- Course website
 - <http://plg.whu.edu.cn:8080/plc/2025sp/>
- Grading
 - Assignments: 10%
 - Midterm exam: 20%
 - Programming assignments: 70%
 - Midterm exam and assignments are expected to be accomplished **independently**.

Course Information

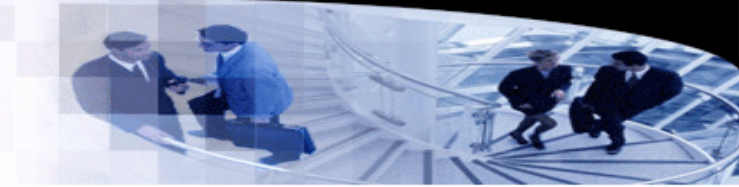


• References

- Benjamin C. Pierce: Types and programming languages. MIT Press 2002, ISBN 978-0-262-16209-8, pp. I-XXI, 1-623
- Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman: Compilers: Principles, Techniques, and Tools. Addison-Wesley series in computer science / World student series edition, Addison-Wesley 1986, ISBN 0-201-10088-6, pp. I-X, 1-796



Programming Languages & Compilers



- Source code vs. machine code

```
#include <iostream>
```

```
int main() {
```

```
    std::wcout << L"Hello world!" << std::endl;
```

```
    return 0;
```

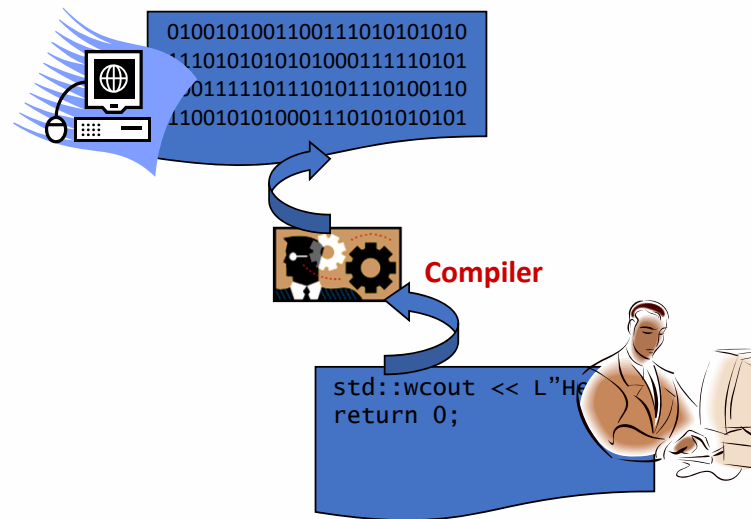
```
}
```

```
main:
004114B0 55             push    ebp
004114B1 8B EC         mov     ebp, esp
004114B3 81 EC C0 00 00 sub     esp, 0C0h
004114B9 53           push    ebx
004114BA 56           push    esi
004114BB 57           push    edi
004114BC 8D BD 40 FF FF lea     edi, [ebp-0C0h]
004114C2 B9 30 00 00 00 mov     ecx, 30h
004114C7 B8 CC CC CC CC mov     eax, 0CCCCCCCCh
004114CC F3 AB         rep stos dword ptr es: [edi]
004114CE 8B F4         mov     esi, esp
004114D0 A1 38 93 41 00 mov     eax, dword ptr [__imp_std::endl (419338h)]
004114D5 50           push    eax
004114D6 68 00 68 41 00 push    offset string L"Hello world!" (416800h)
004114DB 8B 0D 3C 93 41 00 mov     ecx, dword ptr [__imp_std::wcout (41933Ch)]
004114E1 51           push    ecx
004114E2 E8 36 FC FF FF call    std::operator<<<wchar_t, std::char_traits<wchar_t> > (41111Dh)
004114E7 83 C4 08         add     esp, 8
004114EA 8B C8         mov     ecx, eax
004114EC FF 15 20 93 41 00 call    dword ptr [__imp_std::basic_ostream<wchar_t, std::char_traits<wchar_t> >::operator<< (419320h)]
004114F2 3B F4         cmp     esi, esp
004114F4 E8 A6 FC FF FF call    @ILT+410 (__RTC_CheckEsp) (41119Fh)
004114F9 33 C0         xor     eax, eax
004114FB 5F           pop     edi
004114FC 5E           pop     esi
004114FD 5B           pop     ebx
004114FE 81 C4 C0 00 00 00 add     esp, 0C0h
00411504 3B EC         cmp     ebp, esp
00411506 E8 94 FC FF FF call    @ILT+410 (__RTC_CheckEsp) (41119Fh)
0041150B 8B E5         mov     esp, ebp
0041150D 5D           pop     ebp
0041150E C3           ret
```

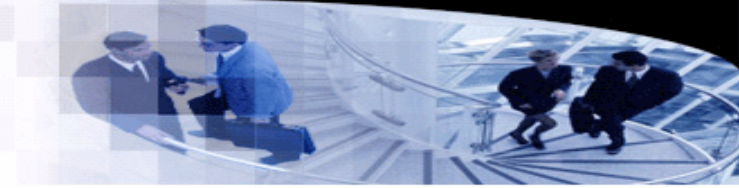
Programming Languages & Compilers

- **Compilers**

- Translating **source** codes into machine codes (or **target** codes in another programming language)



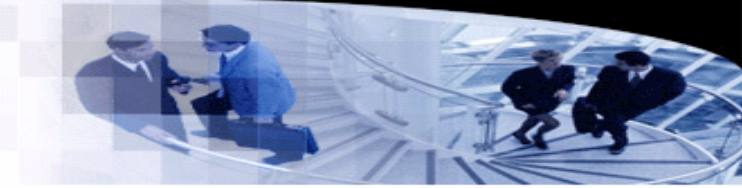
Programming Languages & Compilers



- How it works?

- A typical compiler may be composed of the following components.
 - Preprocessor
 - Scanner (Lexical Analyzer)
 - Parser (Syntax Analyzer)
 - Code Generator
 - Optimizer
 - Target Code Generator

Programming Languages & Compilers



- How it works?

- **Preprocessors** translate strings into strings by handling macros, directives, etc.

```
#include <iostream>
L
int main()
{
    std::wcout << L"Hello world!" << std::endl;
    return 0;
}
```

```
operator>>(basic_istream<wchar_t, char_traits<wchar_t>> &, wchar_t *);
template __declspec(dllimport) basic_istream<wchar_t, char_traits<wchar_t>> >& __cdecl
operator>>(basic_istream<wchar_t, char_traits<wchar_t>> &, wchar_t&);
}
#pragma warning(pop)
#pragma pack(pop)
#pragma pack(push, 8)
#pragma warning(push, 3)

namespace std {
extern __declspec(dllimport) istream *_Ptr_cin;
extern __declspec(dllimport) ostream *_Ptr_cout;
extern __declspec(dllimport) ostream *_Ptr_cerr;
extern __declspec(dllimport) ostream *_Ptr_clog;

class __declspec(dllimport) _Winit {
public:
    __thiscall _Winit();
    __thiscall ~_Winit();
private:
    static int _Init_cnt;
};

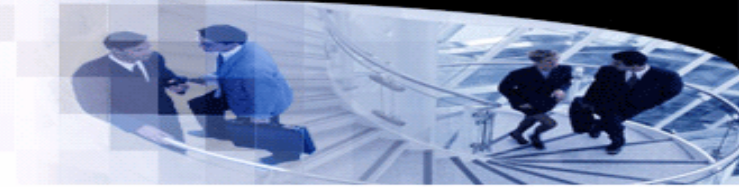
extern __declspec(dllimport) wistream *_Ptr_wcin;
extern __declspec(dllimport) wostream *_Ptr_wcout;
extern __declspec(dllimport) wostream *_Ptr_wcerr;
extern __declspec(dllimport) wostream *_Ptr_wclog;

extern __declspec(dllimport) istream cin;
extern __declspec(dllimport) ostream cout;
extern __declspec(dllimport) ostream cerr;
extern __declspec(dllimport) ostream clog;

extern __declspec(dllimport) wistream wcin;
extern __declspec(dllimport) wostream wcout;
extern __declspec(dllimport) wostream wcerr;
extern __declspec(dllimport) wostream wclog;

}
#pragma warning(pop)
#pragma pack(pop)
int main()
{
    std::wcout << L"Hello world!" << std::endl;
    return 0;
}
```

Programming Languages & Compilers

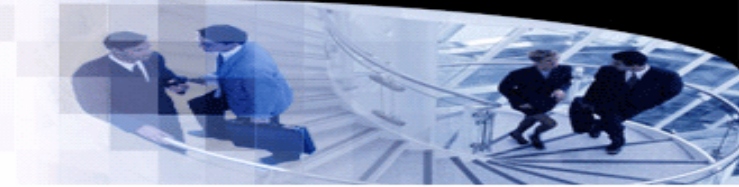


- How it works?

- A **scanner** translates a string (source code) into a sequence of tokens (words).

```
#include <iostream>
int main ( ) {
    std::wcout << L"Hello world!" << std::endl;
    return 0;
}
```

Programming Languages & Compilers



- How it works?

- A **parser** analyzes the input tokens, generates the structure of the source code, and checks semantical errors (type checking).

```
#include <iostream>
int main() {
    std::wcout << L"Hello world!" << std::endl;
    return 0;
}
```

